



Instituto Tecnológico y de Estudios Superiores de Monterrey

Campus Estado de México

Revisión 3 - Avance al 60%

Luis David Pozos Tamez A01800657

Adrián Proaño Bernal A01752615

Juan Pablo Solís Gómez A01800430

Israel González Huerta A01751433

Modelación de Sistemas Multiagentes con Gráficas Computacionales (Grupo 302)

Profesores:

Jorge Adolfo Ramírez Uresti

Mauricio Bezares Peñúñuri

Jueves 27 de noviembre del 2025

Índice

Descripción del medio ambiente.....	2
Diagramas de Agentes.....	3
Diagramas de Organización.....	4
Diagramas de Protocolos de Interacción.....	5
Video de la simulación corriendo.....	6
Código implementación de los agentes.....	6
Código implementación Gráfica.....	14
Plan de trabajo.....	20
Aprendizajes adquiridos.....	21

Descripción del medio ambiente

Inaccesible (60%): Es un medio ambiente inaccesible. Ningún agente tiene un conocimiento completo sobre el medio ambiente y otros agentes. El agente con mayor conocimiento sería el organizador de rutas, el cual conoce la ubicación y destino de los transeúntes y vehículos, junto con una representación del mapa; lo único que desconoce es la ubicación y estado de los semáforos.

No determinista (20%): En teoría, las rutas de cualquier lugar a cualquier otro lugar serían replicables dado que siempre siguen el mismo algoritmo de búsqueda, pero los agentes tienen la posibilidad de ignorarla y por lo no se puede predecir la conducta. Adicionalmente, los transeúntes también pueden elegir entre usar un vehículo propio, un autobús o un taxi.

No episódico (80%): Se puede decir que la conducta de todos los agentes ocurre en pequeños episodios de procesamiento, pero estos episodios no ocurren de manera coordinada para los agentes. Cada agente constantemente está moviéndose y procesando lo que hará a continuación, sin que el programa dedique tiempo en bloques para manejar las operaciones de los agentes.

Dinámico (100%): El ambiente no se detiene para que los agentes decidan qué hacer. Constantemente ocurren acciones.

Discreto (100%): El mapa está completamente seccionado en celdas y cada agente tiene una cantidad limitada de acciones que puede realizar

Diagramas de Agentes

Automóvil	Taxi	Autobús	Transeúnte
Grupo: vehículos	Grupo: vehículos	Grupo: vehículos	Grupo: personas
Rol: vehículo privado	Rol: taxi servicio	Rol: transporte colectivo	Rol: ciudadano
Servicio:	Servicio:	Servicio:	Servicio:
Circular por la ciudad	Transporte bajo demanda	Transporte en ruta fija	Moverse por la ciudad
Protocolo:	Protocolo:	Protocolo:	Protocolo:
Circulación Urbana	Realizar viaje	Recorrer ruta fija	Movilidad peatonal, Movilidad en vehículo
Eventos:	Eventos:	Eventos:	Eventos:
Congestión detectada, Ruta asignada	Solicitud de pasajero, Ruta asignada	Llegada a parada, Alta demanda	Semáforo peatonal, Llegada de vehículo, Parada de autobús
Metas: Llegar al destino, Evitar congestiones	Metas: Recoger pasajero, Llegar al destino, Evitar congestiones	Metas: Completar la ruta	Metas: Llegar a destino, Abordar transporte adecuado
Plan: Elegir mejor ruta	Plan: Elegir mejor ruta, Seleccionar zonas con demanda	Plan: Detenerse en paradas	Plan: Escoger un vehículo
Acciones: Avanzar, Detenerse, Cambiar de ruta, Estacionarse, Recoger pasajero, Dejar pasajero	Acciones: Recoger pasajero, Dejar pasajero, Cambiar de ruta	Acciones: Iniciar recorrido, Avanzar, Detenerse, Recoger pasajero, Dejar pasajero	Acciones: Caminar, Detenerse, Abordar vehículo, Descender de vehículo, Solicitar taxi
Conocimiento:	Conocimiento:	Conocimiento:	Conocimiento:
Mapa vial, Estado del tráfico, Disponibilidad de estacionamiento	Mapa vial, Estado del tráfico, Tarifas, Zonas de alta demanda, Capacidad actual	Ruta asignada, Capacidad actual	Mapa vial, Pasos peatonales, Semáforos, Paradas de autobús

CoordinadorRutas	CoordinadorSemáforos	Semáforo	Estacionamiento
Grupo: coordinadores	Grupo: coordinadores	Grupo: infraestructura vial	Grupo: infraestructura vial
Rol: gestor de rutas	Rol: gestor de semáforos	Rol: regulador de flujo vehicular	Rol: regulador de flujo vehicular
Servicio:	Servicio:	Eventos:	Eventos:
Asignar rutas	Controlar estado de semáforos	Temporizador expirado	Llegada de Vehículo, Salida de Vehículo, Ocupación máxima
Protocolo:	Protocolo:	Evento-Acción:	Evento-Acción:
Coordinar rutas	Coordinar semáforos	Temporizador expirado -> Cambiar al siguiente estado/color (rojo, amarillo o verde)	Llegada de Vehículo -> Actualizar ocupación actual
Eventos:	Eventos:		Salida de Vehículo -> Liberar espacio
Congestión vial, Solicitud de ruta (taxi)	Congestión en cruce		Ocupación máxima -> Marcar estacionamiento como ocupado
Metas: Minimizar tráfico, equilibrar flujo vehicular	Metas: Reducir tiempo de espera, Mantener el flujo en intersecciones		
Plan: Calcular ruta, Redistribuir carga vial	Plan: Sincronizar semáforos		
Acciones: Asignar ruta	Acciones: Cambiar temporizadores de semáforos		
Conocimiento:	Conocimiento:		
Mapa vial, Estado del tráfico	Estado de semáforos, Estado del tráfico		

Figura 1. Diagramas de agentes pensados para el proyecto

Diagramas de Organización

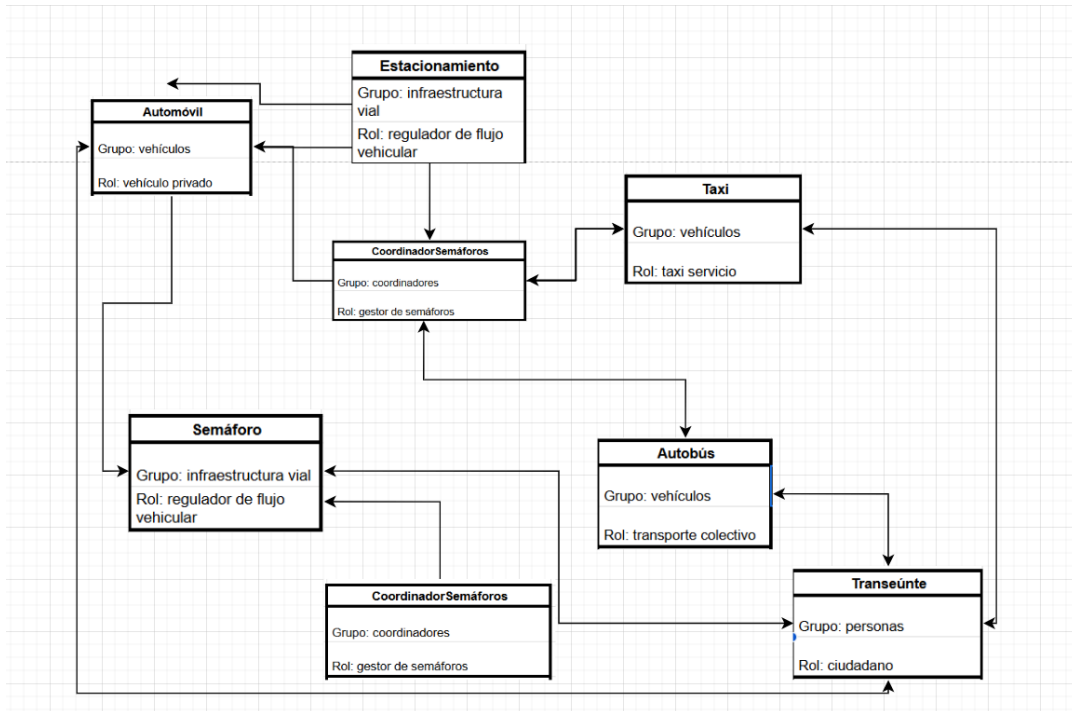


Figura 2. Diagrama de organización de agentes

Diagramas de Protocolos de Interacción

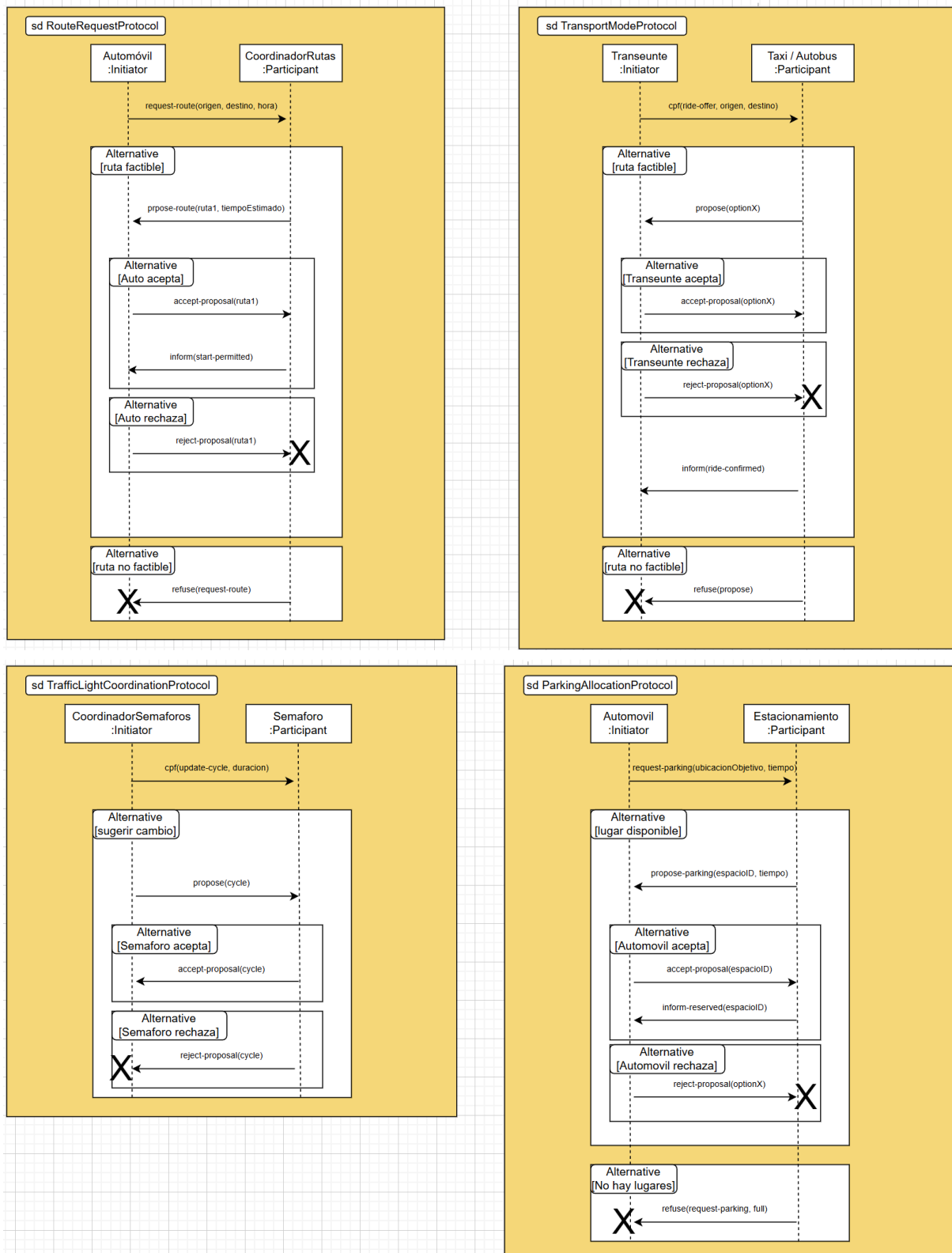


Figura 3. Diagramas de protocolos de interacción entre agentes

Video de la simulación corriendo

Video: https://youtu.be/2a4UQoZ_f7E

***Nota:** Para la prueba en el video se permite que se dé un step cada segundo. La idea es reducir este tiempo en futuras iteraciones para que la simulación se vea más fluida. La prueba en el video se hace con 25 agentes.

Código implementación de los agentes

A partir de la lógica usada para la actividad integradora, se implementó la conexión con Unity a través de un servidor de web socket. El estado completo de la simulación (posición y tipo de cada agente) se envía continuamente para que Unity pueda visualizar la simulación en tiempo real.

```
Python
# type: ignore
import mesa
from mesa.discrete_space import CellAgent, OrthogonalVonNeumannGrid
from collections import deque
import asyncio
import websockets
import json
import diccionario_movimientos

#Funcion "breadth_first_search" que llamaremos para calcular la ruta que
# debe tomar el agente "Carro" para llegar de un punto a otro
# Esta misma funcion se llamara nuevamente en caso de que el agente Carro
# encuentre un obstaculo.

def breadth_first_search(start: tuple, goal: tuple, grid:
OrthogonalVonNeumannGrid) -> list[tuple]:
    queue: deque[tuple] = deque()
    visited: set[tuple] = set()
    parent: dict[tuple, tuple] = {}

    queue.append(start)
    visited.add(start)

    while queue:
        current: tuple = queue.popleft()
        if current == goal:
            path = []
            while current in parent:
                path.append(current)
                current = parent[current]
            path.append(start)
```

```

        path.pop(-1)
        return path[::-1]

    cell = grid[current].coordinate
    neighbors = diccionario_movimientos.movimientos_posibles[cell[0] +
1][cell[1] + 1]

    for i in neighbors:
        cell_pos = (i[0] - 1, i[1] - 1)
        if cell_pos not in visited:
            neighbor_cell = grid[cell_pos]
            has_car = any(isinstance(agent, Carro) for agent in
neighbor_cell.agents)
            is_park = any(isinstance(agent, AgenteCalle) and
agent.isEstacionamiento for agent in neighbor_cell.agents)
            if not has_car or is_park:
                visited.add(cell_pos)
                parent[cell_pos] = current
                queue.append(cell_pos)

    return []

```

Clase "AgenteCalle" que tenga valores de dirección asociados, para marcar la circulación.

```

class AgenteCalle(CellAgent):
    def __init__(self, model, cell):
        super().__init__(model)
        self.cell = cell
        self.isBuilding = False
        self.isEstacionamiento = False

    def step(self):
        pass

```

Agente "Semaforo"

```

class Semaforo1(CellAgent):
    def __init__(self, model, cell):
        super().__init__(model)
        self.cell = cell
        self.avanza = False
        self.contador = 0

    def cambair_estado(self):
        self.avanza = not self.avanza

    def step(self):
        self.contador += 1

```

```

        if self.contador >= 5:
            self.cambair_estado()
            self.contador = 0

class Semaforo2(CellAgent):
    def __init__(self, model, cell):
        super().__init__(model)
        self.cell = cell
        self.avanza = True
        self.contador = 0

    def cambair_estado(self):
        self.avanza = not self.avanza

    def step(self):
        self.contador += 1
        if self.contador >= 5:
            self.cambair_estado()
            self.contador = 0

# Agente "Carro"

class Carro(CellAgent):
    def __init__(self, model, cell):
        super().__init__(model)
        self.cell = cell
        self.destino =
model.random.choice(model.estacionamientos_cells).coordinate
        while self.destino == self.cell.coordinate:
            self.destino =
model.random.choice(model.estacionamientos_cells).coordinate
        self.estacionado = False
        self.ruta = []

    def calcular_ruta(self):
        self.ruta = breadth_first_search(self.cell.coordinate, self.destino,
self.model.grid)

    def puede_avanzar(self):
        siguiente_celda = self.obtener_siguiete()
        if not siguiente_celda:
            return False
        estado_estacionamiento = None
        for agente in siguiente_celda.agents:
            if isinstance(agente, AgenteCalle):
                estado_estacionamiento = agente
            if isinstance(agente, Carro) and not
estado_estacionamiento.isEstacionamiento:

```

```

        return False
    if isinstance(agente, (Semaforo1, Semaforo2)):
        if not agente.avanza:
            return False
    return True

def avanzar(self):
    siguiente_celda = self.obtener_siguiente()
    if siguiente_celda:
        self.cell = siguiente_celda
        self.ruta.pop(0)

def detenerse(self):
    pass

def obtener_siguiente(self):
    if not self.ruta:
        return None
    siguiente_pos = self.ruta[0]
    siguiente_celda = self.model.grid[siguiente_pos]
    return siguiente_celda

def cambiar_carril(self):
    x = self.puede_cambiar()
    if x:
        self.ruta = x

def puede_cambiar(self):
    siguiente_celda = self.obtener_siguiente()
    if not siguiente_celda:
        return False
    for agente in siguiente_celda.agents:
        if isinstance(agente, (Semaforo1, Semaforo2)):
            return False
    return breadth_first_search(self.cell.coordinate, self.destino,
self.model.grid)

def llego_destino(self):
    return self.cell.coordinate == self.destino

def estacionarse(self):
    self.estacionado = True

def step(self):
    if not self.ruta:
        self.calcular_ruta()
    if not self.ruta:

```

```

        self.destino =
self.model.random.choice(self.model.estacionamientos_cells).coordinate
        while self.destino == self.cell.coordinate:
            self.destino =
self.model.random.choice(self.model.estacionamientos_cells).coordinate
        if not self.estacionado:
            if self.puede_avanzar():
                self.avanzar()
            else:
                self.cambiar_carril()
        if self.llego_destino():
            self.estacionarse()
        if self.estacionado:
            self.remove()

# Modelo

class TrafficModel(mesa.Model):
    def __init__(self, n):
        super().__init__()
        self.num_cars = n
        self.grid = OrthogonalVonNeumannGrid((24, 24), torus = False,
capacity = 100, random = self.random)
        self.estacionamientos_cells = []

    for x in range(width):
        for y in range(height):
            cell = self.grid[(x, y)]
            calle = AgenteCalle(self, cell)

#Set Semaforos1
    if (x == 21 and (y == 4 or y == 5 or y == 10 or y == 11) or
        x == 2 and (y == 4 or y == 5 or y == 8 or y == 9) or
        x == 7 and (y == 22 or y == 23) or
        x == 15 and (y == 22 or y == 23)):
        semaforo1 = Semaforo1(self, cell)

#Set Semaforos2
    if (y == 3 and (x == 0 or x == 1) or
        y == 7 and (x == 0 or x == 1) or
        y == 2 and (x == 10 or x == 11) or
        y == 6 and (x == 22 or x == 23) or
        y == 12 and (x == 22 or x == 23) or
        y == 21 and (x == 8 or x == 9 or x == 16 or x == 17)):
        semaforo2 = Semaforo2(self, cell)

#Set Buildings
    if ((1 < x < 4) and (1 < y < 4) or

```

```

        (1 < x < 4) and (5 < y < 8) or
        (1 < x < 4) and (11 < y < 16) or
        (1 < x < 8) and (17 < y < 22) or
        (5 < x < 8) and (1 < y < 4) or
        (5 < x < 8) and (5 < y < 8) or
        (5 < x < 8) and (11 < y < 16) or
        (11 < x < 22) and (1 < y < 4) or
        (11 < x < 22) and (5 < y < 8) or
        (11 < x < 16) and (11 < y < 15) or
        (11 < x < 16) and (16 < y < 22) or
        (17 < x < 22) and (11 < y < 22) or
        (8 < x < 11) and (8 < y < 11)):
    calle.isBuilding = True

    #Set Estacionamiento
    if ((x == 3 and y == 3) or # Estacionamiento 13
        (x == 7 and y == 6) or # Estacionamiento 17
        (x == 3 and y == 12) or # Estacionamiento 12
        (x == 6 and y == 15) or # Estacionamiento 15
        (x == 6 and y == 18) or # Estacionamiento 16
        (x == 4 and y == 21) or # Estacionamiento 14
        (x == 13 and y == 3) or # Estacionamiento 2
        (x == 14 and y == 6) or # Estacionamiento 5
        (x == 14 and y == 14) or # Estacionamiento 3
        (x == 12 and y == 18) or # Estacionamiento 1
        (x == 14 and y == 21) or # Estacionamiento 4
        (x == 20 and y == 2) or # Estacionamiento 10
        (x == 19 and y == 7) or # Estacionamiento 8
        (x == 19 and y == 12) or # Estacionamiento 7
        (x == 21 and y == 16) or # Estacionamiento 11
        (x == 18 and y == 19) or # Estacionamiento 6
        (x == 20 and y == 21)): # Estacionamiento 9
        calle.isEstacionamiento = True
        self.estacionamientos_cells.append(self.grid[(x, y)])

    agents = Carro.create_agents(
        self,
        self.num_cars,
        self.random.choices(self.estacionamientos_cells,
k=self.num_cars),

    )
    def step(self):
        for agent in list(self.agents):
            agent.step()

# --- LÓGICA WEBSOCKET ---

```

```

connected_clients = set()
width = 24
height = 24
model = TrafficModel(25)

def serialize_agents():
    """Serializa todos los agentes del modelo para enviar a Unity"""
    agents_data = []

    for agent in model.agents:
        agent_dict = {
            "id": agent.unique_id,
            "x": agent.cell.coordinate[0],
            "y": agent.cell.coordinate[1],
            "type": type(agent).__name__
        }

        # Agregar información específica según el tipo de agente
        if isinstance(agent, Carro):
            agent_dict["estacionado"] = agent.estacionado
            agent_dict["destino_x"] = agent.destino[0]
            agent_dict["destino_y"] = agent.destino[1]
        elif isinstance(agent, (Semaforo1, Semaforo2)):
            agent_dict["avanza"] = agent.avanza
        elif isinstance(agent, AgenteCalle):
            agent_dict["isBuilding"] = agent.isBuilding
            agent_dict["isEstacionamiento"] = agent.isEstacionamiento

        agents_data.append(agent_dict)

    return agents_data

async def send_world_state():
    """Envía el estado actual del mundo a todos los clientes conectados"""
    if not connected_clients:
        return

    state = {
        "type": "update",
        "step": model.steps,
        "agents": serialize_agents()
    }
    msg = json.dumps(state)

    # Broadcast a todos los clientes
    websockets.broadcast(connected_clients, msg)

async def simulation_loop():

```

```

"""Loop principal de la simulación"""
while True:
    model.step()
    await send_world_state()
    await asyncio.sleep(1) # Delay entre pasos (ajustable)

async def handler(ws):
    """Maneja las conexiones WebSocket"""
    connected_clients.add(ws)
    print(f"Unity conectado. Total clientes: {len(connected_clients)}")

    try:
        # Enviar estado inicial
        await send_world_state()

        # Mantener la conexión abierta
        async for message in ws:
            # Procesar comandos desde Unity si es necesario
            try:
                data = json.loads(message)
                if data.get("type") == "pause":
                    print("Simulación pausada")
                elif data.get("type") == "resume":
                    print("Simulación reanudada")
            except json.JSONDecodeError:
                print(f"Mensaje inválido recibido: {message}")

    except websockets.ConnectionClosed:
        print("Unity desconectado.")
    finally:
        connected_clients.remove(ws)
        print(f"Clientes restantes: {len(connected_clients)}")

if __name__ == "__main__":
    async def main():
        # Iniciar servidor WebSocket
        server = await websockets.serve(handler, "localhost", 8765)
        print("Servidor Mesa listo en ws://localhost:8765")
        print(f"Modelo inicializado con {model.num_cars} carros")
        print(f"Grid: {width}x{height}")

        # Iniciar loop de simulación
        asyncio.create_task(simulation_loop())

        # Mantener el servidor corriendo
        await asyncio.get_running_loop().create_future()

    try:

```

```
        asyncio.run(main())
    except KeyboardInterrupt:
        print("\nServidor detenido.")
```

Código implementación Gráfica

Para la parte gráfica, este script se asigna a un game object vacío. El script se conecta al mismo web socket que el servidor de mesa y recibe en tiempo real el estado de todos los agentes de la simulación (carros, semáforos, calles, edificios y estacionamientos) y los refleja en Unity instanciando prefabs y actualizando sus posiciones. Según el tipo de agente y sus propiedades (por ejemplo, si el semáforo está en verde o rojo, o si el carro ya está estacionado), ajusta su apariencia en la escena, rota los carros en la dirección de movimiento y elimina los objetos que dejan de existir en la simulación.

```
CSharp
using UnityEngine;
using NativeWebSocket;
using Newtonsoft.Json.Linq;
using System.Collections.Generic;
using System.Text;

public class MesaSync : MonoBehaviour
{
    WebSocket ws;

    [Header("Configuración de Conexión")]
    public string serverUrl = "ws://localhost:8765";

    [Header("Prefabs por Tipo de Agente")]
    public GameObject carroPrefab;
    public GameObject semaforo1Prefab;
    public GameObject semaforo2Prefab;
    public GameObject callePrefab;
    public GameObject buildingPrefab;
    public GameObject estacionamientoPrefab;

    // Diccionario local para mantener referencia de los objetos
    instanciados
```

```

    private Dictionary<int, GameObject> unityAgents = new Dictionary<int,
GameObject>();

    // Diccionario para guardar la posición anterior de cada carro
    private Dictionary<int, Vector3> previousPositions = new Dictionary<int,
Vector3>();

    async void Start()
    {
        ws = new WebSocket(serverUrl);

        ws.OnOpen += () => Debug.Log("Conexión establecida con Mesa.");
        ws.OnError += (e) => Debug.LogError("Error en WebSocket: " + e);
        ws.OnClose += (e) => Debug.Log("Conexión cerrada: " + e);

        ws.OnMessage += (bytes) =>
        {
            string message = Encoding.UTF8.GetString(bytes);

            try
            {
                JObject json = JObject.Parse(message);

                if ((string)json["type"] == "update")
                {
                    JArray agentsData = (JArray)json["agents"];
                    UpdateUnityScene(agentsData);
                }
            }
            catch (System.Exception ex)
            {
                Debug.LogWarning("Error procesando JSON: " + ex.Message);
            }
        };

        await ws.Connect();
    }

    void UpdateUnityScene(JArray mesaAgents)
    {
        HashSet<int> activeAgents = new HashSet<int>();

        foreach (var agentData in mesaAgents)
        {
            int id = (int)agentData["id"];
            float x = (float)agentData["x"];
            float y = (float)agentData["y"];
            string agentType = (string)agentData["type"];

```

```

activeAgents.Add(id);

// Si el agente no existe en Unity, lo creamos
if (!unityAgents.ContainsKey(id))
{
    GameObject prefab = GetPrefabForType(agentType, agentData);

    if (prefab != null)
    {
        GameObject newAgent = Instantiate(prefab);
        newAgent.name = $"{agentType}_{id}";
        unityAgents[id] = newAgent;

        // Configurar propiedades específicas
        ConfigureAgent(newAgent, agentType, agentData);
    }
    else
    {
        Debug.LogWarning($"No hay prefab asignado para tipo:
{agentType}");
        continue;
    }
}

// Actualizamos la posición (Mesa usa Y, Unity usa Z para el
plano)
Vector3 targetPosition = new Vector3(x, 0, 23-y);

// Rotar el carro hacia la dirección del movimiento
if (agentType == "Carro")
{
    if (previousPositions.ContainsKey(id))
    {
        Vector3 direction = targetPosition -
previousPositions[id];

        // Solo rotar si hay movimiento significativo
        if (direction.magnitude > 0.01f)
        {
            // Calcular el ángulo basado en la dirección
            float angle = 0f;

            if (Mathf.Abs(direction.x) > Mathf.Abs(direction.z))
            {
                // Movimiento horizontal (Este u Oeste)
                angle = direction.x > 0 ? 90f : -90f;
            }
        }
    }
}

```

```

        else
        {
            // Movimiento vertical (Norte o Sur)
            angle = direction.z > 0 ? 0f : 180f;
        }

        unityAgents[id].transform.rotation =
Quaternion.Euler(0, angle, 0);
    }
}
// else
// {
//     // Rotación inicial para carros nuevos
//     unityAgents[id].transform.rotation =
Quaternion.Euler(0, 0, 0);
// }

// Guardar la posición actual para el próximo frame
previousPositions[id] = targetPosition;
}

unityAgents[id].transform.position = targetPosition;

// Actualizar propiedades específicas del agente
UpdateAgentProperties(unityAgents[id], agentType, agentData);
}

// Eliminar agentes que ya no existen en Mesa (carros estacionados)
List<int> toRemove = new List<int>();
foreach (var kvp in unityAgents)
{
    if (!activeAgents.Contains(kvp.Key))
    {
        toRemove.Add(kvp.Key);
    }
}

foreach (int id in toRemove)
{
    Destroy(unityAgents[id]);
    unityAgents.Remove(id);
    Debug.Log($"Agente {id} removido (estacionado)");
}
}

GameObject GetPrefabForType(string agentType, JToken agentData)
{
    switch (agentType)

```

```

{
    case "Carro":
        return carroPrefab;
    case "Semaforo1":
        return semaforo1Prefab;
    case "Semaforo2":
        return semaforo2Prefab;
    case "AgenteCalle":
        // Diferenciar entre calle, building y estacionamiento
        bool isBuilding = agentData["isBuilding"] != null ?
(bool)agentData["isBuilding"] : false;
        bool isEstacionamiento = agentData["isEstacionamiento"] !=
null ? (bool)agentData["isEstacionamiento"] : false;

        if (isBuilding)
            return buildingPrefab;
        else if (isEstacionamiento)
            return estacionamientoPrefab;
        else
            return callePrefab;
    default:
        return null;
}
}

void ConfigureAgent(GameObject agent, string agentType, JToken
agentData)
{
    // Configuración inicial específica por tipo
    switch (agentType)
    {
        case "Carro":
            // Añadir componente de movimiento suave si lo deseas
            agent.tag = "Car";
            break;
        case "Semaforo1":
        case "Semaforo2":
            agent.tag = "TrafficLight";
            break;
        case "AgenteCalle":
            agent.tag = "Road";
            bool isBuilding = agentData["isBuilding"] != null ?
(bool)agentData["isBuilding"] : false;
            if (isBuilding)
                agent.tag = "Building";
            break;
    }
}
}

```

```

    void UpdateAgentProperties(GameObject agent, string agentType, JToken
agentData)
    {
        // Actualizar propiedades dinámicas
        switch (agentType)
        {
            case "Carro":
                bool estacionado = agentData["estacionado"] != null ?
(bool)agentData["estacionado"] : false;
                // Cambiar color o material si está estacionado
                if (estacionado)
                {
                    var renderer = agent.GetComponent<Renderer>();
                    if (renderer != null)
                        renderer.material.color = Color.gray;
                }
                break;

            case "Semaforo1":
            case "Semaforo2":
                bool avanza = agentData["avanza"] != null ?
(bool)agentData["avanza"] : false;
                // Cambiar color del semáforo
                var lightRenderer = agent.GetComponent<Renderer>();
                if (lightRenderer != null)
                {
                    lightRenderer.material.color = avanza ? Color.green :
Color.red;
                }
                break;
        }
    }

    // Método para enviar comandos a Mesa
    public async void SendCommandToMesa(string command)
    {
        if (ws.State == WebSocketState.Open)
        {
            JObject payload = new JObject
            {
                ["type"] = command
            };

            await ws.SendText(payload.ToString());
        }
    }
}

```

```

    void Update()
    {
        #if !UNITY_WEBGL || UNITY_EDITOR
            ws?.DispatchMessageQueue();
        #endif
    }

    private async void OnApplicationQuit()
    {
        if (ws != null) await ws.Close();
    }
}

```

Plan de trabajo

*Sujeto a cambios según el avance del proyecto

Actividad	Descripción	Responsables principales	Fecha estimada	Estado	Esfuerzo
Diseño conceptual del SMA	Definir agentes, reglas, variables, entorno urbano simulado	Luis David, Adrián	14–18 nov	Finalizado	6–10 h
Implementación base en Mesa	Crear agentes, grid, simulación básica de movimiento	Luis David, Adrián	18–25 nov	Finalizado	10–14 h
Modelado 3D del entorno	Construcción de calles, avenidas y edificios en Unity	Juan Pablo, Israel	18–30 nov	En progreso	8–12 h
Integración Mesa → Unity	Importar datos, sincronizar posiciones	Todos	27 nov – 1 dic	En progreso	8–12 h

Animación 3D de vehículos	Movimiento según datos, rotaciones, caminos	Israel, Juan Pablo	28 nov – 2 dic	Pendiente	10–15 h
Implementación de estrategias	Rutas alternas, semáforos inteligentes, carpooling	Luis David, Adrián	25 nov– 3 dic	En progreso	12–18 h
UI básica y cámara	Visualización final, cámara orbitante, UI mínima	Israel, Juan Pablo	2– 4 dic	Pendiente	4–6 h
Documentación	Arquitectura, descripción del modelo	Todos	27 nov – 4 dic	En progreso	8–12 h
Presentación final	Demo + explicación del sistema	Todos	4– 5 dic	Pendiente	3–5 h

Aprendizajes adquiridos

Israel González Huerta

Aprendí sobre la clasificación de sistemas multiagentes y la teoría de sistemas. En la práctica, entendí la utilidad de tratar a elementos de un programa como agentes con el fin de generar simulaciones. Por su parte, ahora conozco lo básico de la modelación en 3D y apliqué ese conocimiento para comenzar a modelar la ciudad y así poder representar la simulación.

Juan Pablo Solís Gómez

Aprendí sobre la importancia de la colaboración de los agentes. Entendí cómo hacer la comunicación entre Mesa y Unity, para que el comportamiento programado de los agentes se traduzca a lo visual. Aprendí a modelar en Unity con la herramienta ProBuilder, lo cual me sirvió para experimentar con las texturas y todas las opciones de personalización que ofrece este paquete, para así poder modelar un mapa lo más similar posible al que se tenía para la Actividad Integradora, y así poder implementar la misma lógica para la conexión con Unity.

Adrian Proaño Bernal

Aprendí sobre la lógica detrás de las decisiones de un agente reactivo, analizando los diferentes escenarios en los cuales se puede encontrar, y tener una solución clara y concisa sobre cómo reaccionaría en esos escenarios. Aprendí a tener un pensamiento más lógico para poder solucionar los posibles obstáculos que los agentes se pudieran encontrar. Finalmente, aprendí a implementar mi conocimiento sobre estructura de datos utilizando la herramienta de Mesa, para poder crear una simulación que pudiera correr de forma rápida y eficaz.

Luis David Pozos Tamez

Aprendí acerca de la arquitectura y lógica detrás de la conexión y sincronización entre diferentes lenguajes y herramientas (en este caso, la librería Mesa de Python y la visualización en Unity). Durante este proceso surgieron todo tipo de problemas inesperados, los cuales me hicieron darme cuenta de la importancia de prestar atención a los detalles y plantear las interacciones con los agentes y el modelo de una manera correcta.